



Instituto Politécnico do Cávado e do Ave
Escola Superior de Tecnologia

BookSwap IPCA

Plataforma de Troca e Partilha de Livros

Relatório de Trabalho Prático

Armazenamento e Acesso a Dados

Alunos:

N.º 37161 – Juan Mathias Palacios

N.º 35305 – Simão Filipe Fernandes Costa

N.º 35308 – David Mauri Peixoto Leite

N.º 31891 – Bruno Antunes

Professora: Marta Martinho

Curso Técnico Superior Profissional de Aplicações Móveis

Guimaraes, maio, 2026

Resumo

O presente relatório descreve o processo de análise, desenho e implementação de um sistema de gestão de base de dados para a plataforma BookSwap IPCA, uma aplicação móvel dedicada à troca, empréstimo e doação de manuais escolares e livros técnicos entre estudantes do Instituto Politécnico do Cávado e do Ave (IPCA).

A metodologia de desenvolvimento seguiu as etapas clássicas de engenharia de bases de dados: análise de requisitos, modelação concetual (Diagrama Entidade-Relacionamento), modelação lógica com normalização até à 3.^a Forma Normal (3FN) e implementação física em SQL Server, incluindo a definição de scripts de criação da base de dados, inserção de dados de teste, stored procedures, funções, vistas e triggers.

Como resultado, obteve-se uma base de dados funcional e normalizada, capaz de garantir a consistência e integridade dos dados, suportando todas as operações críticas da plataforma, desde a gestão de utilizadores e catálogo de livros até ao fluxo completo de trocas e comunicação entre utilizadores.

Palavras-Chave:

- Base de Dados Relacional
- SQL Server
- Modelação de Dados
- BookSwap
- T-SQL
- Normalização
- 3FN (Terceira Forma Normal)
- Diagrama Entidade-Relacionamento (DER)
- Stored Procedures
- Triggers
- Vistas
- User-Defined Functions (UDF)
- Entity Framework Core

Lista de Abreviaturas e Siglas

BD	Base de Dados
SGBD	Sistema de Gestão de Base de Dados
SQL	Structured Query Language
T-SQL	Transact-SQL (extensão SQL da Microsoft)
ER / DER	Entidade-Relacionamento / Diagrama Entidade-Relacionamento
PK	Primary Key (Chave Primária)
FK	Foreign Key (Chave Estrangeira)
1FN / 2FN / 3FN	Primeira / Segunda / Terceira Forma Normal
SSMS	SQL Server Management Studio
SP	Stored Procedure
UDF	User-Defined Function
EF Core	Entity Framework Core (ORM utilizado na API)
API	Application Programming Interface
ISBN	International Standard Book Number
IPCA	Instituto Politécnico do Cávado e do Ave

Índice

1. Introdução	1
1.1. Contextualização	1
1.2. Motivação e Objetivos	1
1.3. Estrutura do Documento	2
2. Desenho Concetual da Base de Dados	3
2.1. Requisitos do Sistema	3
2.2. Identificação de Entidades	4
2.3. Identificação de Relacionamentos	5
2.4. Definição de Atributos	6
2.5. Identificação de Chaves Primárias e Candidatas	9
2.6. Diagrama do Modelo Concetual Final	10
3. Desenho Lógico da Base de Dados	11
3.1. Derivação dos Relacionamentos para o Modelo Lógico	11
3.2. Validação do Modelo com Normalização	13
3.3. Validação do Modelo com Transações do Utilizador	16
3.4. Diagrama do Modelo Lógico Final	18
4. Desenho Físico da Base de Dados	19
4.1. Tradução do Modelo Lógico para o Modelo Físico	19
4.2. Índices	23
4.3. Scripts SQL de Criação da Base de Dados	24
4.4. Scripts SQL de Inserção de Dados	27
4.5. Stored Procedures	29
4.6. Funções (UDF)	31
4.7. Vistas (Views)	33
4.8. Triggers	35
5. Conclusão	38
Bibliografia	

1. Introdução

1.1. Contextualização

O presente trabalho surge no âmbito da unidade curricular de Armazenamento e Acesso a Dados, inserida no Curso Técnico Superior Profissional de Aplicações Móveis do Instituto Politécnico do Cávado e do Ave (IPCA). O projeto foca-se no desenvolvimento de uma solução tecnológica para a gestão de uma plataforma de economia circular — o BookSwap IPCA —, destinada a facilitar a troca, empréstimo e doação de manuais escolares e livros técnicos entre a comunidade estudantil.

Atualmente, a partilha de recursos didáticos entre alunos ocorre de forma maioritariamente informal, através de grupos em redes sociais ou contactos diretos. Esta informalidade acarreta problemas como a ausência de um inventário centralizado, a dificuldade no rastreamento de pedidos e empréstimos, e a incapacidade da instituição extrair indicadores sobre a reutilização de manuais.

Neste contexto, este documento descreve o processo completo de desenho e implementação da base de dados em SQL Server, que serve de suporte a todas as operações da plataforma, garantindo a integridade, segurança e persistência dos dados. A API foi desenvolvida em ASP.NET Core 9 com Entity Framework Core, e a aplicação cliente em Flutter.

1.2. Motivação e Objetivos

Motivação

A motivação para o desenvolvimento deste sistema assenta em três pilares fundamentais:

- **Sustentabilidade e Economia Circular:** promover a reutilização de manuais escolares dentro do IPCA, reduzindo o desperdício e o impacto ambiental associado à produção de novos livros.
- **Responsabilidade Social:** apoiar os estudantes com maiores dificuldades económicas, facilitando o acesso gratuito ou a baixo custo a materiais de estudo essenciais.
- **Consolidação Técnica:** aplicar de forma integrada os conhecimentos de modelação de dados, normalização e álgebra relacional num cenário real, garantindo que a base de dados suporte eficientemente uma aplicação móvel e uma API REST.

Objetivos

O objetivo geral é desenhar e implementar uma base de dados relacional robusta em SQL Server para gerir o inventário e as transações de livros. Como objetivos específicos:

- **Modelar o domínio:** identificar com precisão as doze entidades do modelo conceitual (Users, Books, UserBooks, Transactions, Conversations, Messages, AppSettings, Authors, Categories, BookAuthors, BookCategories e AuthTokens) e os seus relacionamentos.
- **Garantir a integridade dos dados:** aplicar regras de normalização até à 3.ª Forma Normal para eliminar redundâncias e evitar anomalias de atualização.
- **Otimizar a consulta:** validar a estrutura lógica através de álgebra relacional, garantindo que o sistema responde eficazmente a pesquisas complexas.
- **Implementar a programação da base de dados:** criar stored procedures, funções, vistas e triggers que encapsulem a lógica de negócio.

- Suportar a escalabilidade: desenhar uma estrutura que permita o crescimento do número de utilizadores e registos sem perda de performance.

1.3. Estrutura do Documento

Este relatório está organizado nos seguintes capítulos:

- Capítulo 1 – Introdução: apresenta a contextualização, motivação e objetivos do projeto.
- Capítulo 2 – Desenho Concetual: descreve o levantamento de requisitos, a identificação de entidades, relacionamentos e atributos, culminando no Diagrama Entidade-Relacionamento (DER).
- Capítulo 3 – Desenho Lógico: detalha a conversão para o modelo relacional, a aplicação das regras de normalização (3FN) e a validação através de álgebra relacional.
- Capítulo 4 – Desenho Físico: apresenta a implementação técnica em SQL Server, incluindo scripts SQL de criação, inserção de dados, e objetos de programação (stored procedures, funções, vistas e triggers).
- Capítulo 5 – Conclusão: resume as principais conclusões e perspetivas de desenvolvimento futuro.

2. Desenho Concetual da Base de Dados

2.1. Requisitos do Sistema

O sistema BookSwap IPCA foi concebido para gerir a partilha de recursos académicos entre estudantes. Os requisitos funcionais dividem-se em sete eixos principais:

Gestão de Utilizadores

O sistema deve permitir o registo e autenticação de utilizadores com os atributos Nome, Email, Password (encriptada com BCrypt), Contacto e Morada. O sistema distingue entre utilizadores comuns e administradores através de um atributo booleano IsAdmin.

Gestão do Catálogo de Livros

O sistema mantém um catálogo global de livros identificados pelo ISBN-13, com informação bibliográfica obtida da API OpenLibrary (título, editora, ano de publicação, descrição, URL da capa). Os autores dos livros são geridos numa tabela própria (Authors), ligada a Books através da tabela associativa BookAuthors. Da mesma forma, as categorias temáticas (Categories) classificam os livros através da tabela associativa BookCategories. Cada utilizador regista as suas cópias físicas na tabela UserBooks, especificando o estado de conservação, a quantidade e a disponibilidade.

Gestão de Trocas e Transações

O sistema permite que um utilizador solicite um exemplar a outro. A tabela Transactions documenta todo o fluxo desde o pedido até à conclusão da troca, suportando três tipos de movimento — Compra, Requisição (empréstimo) e Troca Direta — e cinco estados (Pendente, Aceita, Confirmada, Rejeitada, Cancelada). As transações referenciam os exemplares específicos envolvidos através de UserBookId e, no caso de trocas diretas, SenderUserBookId regista o exemplar oferecido pelo remetente.

Sistema de Mensagens

A plataforma inclui um sistema de mensagens assíncronas entre utilizadores. A tabela Conversations agrupa as trocas de mensagens, podendo estar associadas a um livro específico e/ou a uma transação. A tabela Messages regista cada mensagem individual com o seu conteúdo, remetente e estado de leitura.

Autenticação e Sessões

A autenticação dos utilizadores é gerida através da tabela AuthTokens, que armazena tokens hash únicos por sessão, com data de criação e de expiração. Esta tabela permite que cada utilizador tenha múltiplas sessões ativas (por exemplo, telemóvel e desktop), suporta a revogação individual de tokens e a expiração automática das sessões.

Configuração da Aplicação

A plataforma inclui um sistema simples de configurações geríveis pelo administrador. A tabela AppSettings armazena pares chave/valor (por exemplo, AllowDuplicateBooks, ShowSingleBookCondition) que controlam o comportamento transversal do sistema sem necessidade de alteração de código.

2.2. Identificação de Entidades

Nesta fase foram isolados os objetos do mundo real sobre os quais o sistema necessita de manter registos. Para cada entidade apresenta-se uma descrição e um exemplo de ocorrência. Da análise da arquitetura final do código foram identificadas doze entidades, todas implementadas como tabelas na base de dados SQL Server. A relação muitos-para-muitos entre Books e Authors é resolvida pela tabela associativa BookAuthors (conforme a Regra 6 do Doc01 — Atividade 3), e a relação M:N entre Books e Categories pela tabela

BookCategories. O sistema inclui ainda uma tabela AuthTokens dedicada à gestão de sessões e autenticação dos utilizadores.

Entidade	Descrição	Exemplo de Ocorrência
Users	Regista os estudantes e administradores que acedem à plataforma.	{ Id: 1, Name: "Marta Silva", Email: "marta@alunos.ipca.pt", IsAdmin: false }
Books	Catálogo global de livros identificados por ISBN. Funciona como cache de dados da API OpenLibrary.	{ ISBN: "9780132350884", Title: "Clean Code", Publisher: "Prentice Hall" }
UserBooks	Tabela associativa que liga utilizadores ao catálogo de livros, registando os exemplares físicos de cada utilizador.	{ Id: 5, UserId: 1, ISBN: "9780132350884", Condition: "semi-novo", Quantity: 1, Available: true }
Transactions	Documenta o histórico de todas as trocas, compras e requisições, concluídas ou em curso. Liga-se a UserBooks (exemplar transacionado) e, no caso de trocas, ao exemplar do remetente.	{ Id: 100, Type: "Troca", Status: "Confirmada", UserBookId: 5, SenderUserBookId: 9, SenderId: 1, ReceiverId: 3 }
Conversations	Agrupar as conversas entre dois utilizadores, podendo estar associadas a um livro e/ou transação.	{ Id: 10, InitiatorId: 3, ReceiverId: 1, BookISBN: "9780132350884", Status: "Active" }
Messages	Regista cada mensagem individual no âmbito de uma conversa.	{ Id: 200, ConversationId: 10, SenderId: 3, Content: "Olá, está disponível?", IsRead: false }
AppSettings	Guarda parâmetros de configuração global da plataforma, geríveis pelo administrador (chave/valor).	{ SettingKey: "AllowDuplicateBooks", SettingValue: "false", Description: "Permitir duplicados na biblioteca" }
Authors	Catálogo de autores literários. Resolve a normalização do atributo Authors da entidade Books através da relação M:N implementada por BookAuthors. Cada autor pode estar referenciado num catálogo externo (OpenLibrary).	{ Id: 7, Name: "Robert C. Martin", OpenLibraryId: "OL...", Bio: "..." }
Categories	Catálogo de categorias temáticas (géneros) usadas para classificar livros. Substitui o campo Genre desnormalizado anteriormente em Books, através da relação M:N implementada por BookCategories.	{ Id: 3, Name: "Informática" }
BookAuthors	Tabela associativa que resolve a relação muitos-para-muitos entre Books e Authors. Cada linha indica que um determinado autor escreveu um determinado livro. PK composta por (BookISBN, AuthorId).	{ BookISBN: "9780132350884", AuthorId: 7 }
BookCategories	Tabela associativa que resolve a relação muitos-para-muitos entre Books e Categories. Cada linha indica que um determinado livro pertence a uma determinada categoria. PK composta por (BookISBN, CategoryId).	{ BookISBN: "9780132350884", CategoryId: 3 }
AuthTokens	Armazena os tokens de autenticação ativos de cada utilizador (sessões). Permite revogar tokens, gerir múltiplas sessões e implementar expiração de sessão.	{ Id: 42, UserId: 1, TokenHash: "a3f8...", CreatedAt: ..., ExpiresAt: ... }

2.3. Identificação de Relacionamentos

A tabela seguinte apresenta as ligações entre as entidades do sistema, com a notação de cardinalidades que define as regras de negócio da plataforma.

Entidade A	Cardinalidade	Relacionamento	Cardinalidade	Entidade B
Users	1..1	Possui (exemplares)	0..*	UserBooks
Books	1..1	É exemplificado por	0..*	UserBooks
Users	1..1	Mantém (sessões)	0..*	AuthTokens
Books	1..1	Inclui (autor)	0..*	BookAuthors
Authors	1..1	Aparece em	0..*	BookAuthors
Books	1..1	Inclui (categoria)	0..*	BookCategories
Categories	1..1	Classifica	0..*	BookCategories
UserBooks	0..1	É transacionado em	0..*	Transactions
UserBooks	0..1	É oferecido em (Troca)	0..*	Transactions
Users (Sender)	1..1	Envia	0..*	Transactions
Users (Receiver)	1..1	Recebe	0..*	Transactions
Users (Initiator)	1..1	Inicia conversa	0..*	Conversations
Users (Receiver)	1..1	Participa em conversa	0..*	Conversations
Books	0..1	Contextualiza	0..*	Conversations
Transactions	0..1	Está associada a	0..*	Conversations
Conversations	1..1	Contém mensagens	0..*	Messages
Users	1..1	Escreve	0..*	Messages

Notas: (1) A entidade AppSettings é autónoma — guarda parâmetros globais e não estabelece relacionamentos com outras tabelas. (2) A relação Books↔Authors é muitos-para-muitos (M:N) e está resolvida pela tabela associativa BookAuthors, com PK composta por (BookISBN, AuthorId), conforme a Regra 6 do Doc01 — Atividade 3. Da mesma forma, a relação Books↔Categories é M:N e está resolvida pela tabela associativa BookCategories, com PK composta por (BookISBN, CategoryId). (3) A tabela AuthTokens estabelece uma relação 1:N com Users — cada utilizador pode ter múltiplas sessões ativas em simultâneo. (4) As Transactions ligam-se a UserBooks (exemplar concreto) através do campo UserBookId; no caso de trocas diretas, SenderUserBookId regista o exemplar oferecido em troca pelo remetente, refletindo a arquitetura real da API.

2.4. Definição de Atributos

Para cada entidade, listam-se abaixo todos os atributos, especificando o tipo de dados, obrigatoriedade e se o campo pode ser multivalorado.

Entidade: Users

Atributo	Tipo de Dados	Descrição	Nulos	Multi-valor	Restrição
Id	int (Identity)	Identificador único	Não	Não	PK
Name	nvarchar(max)	Nome completo	Não	Não	NOT NULL
Email	nvarchar(max)	Endereço de email	Não	Não	UNIQUE

Password	nvarchar(max)	Hash BCrypt da password	Não	Não	NOT NULL
Phone	nvarchar(max)	Número de contacto	Não	Não	NOT NULL
Address	nvarchar(max)	Morada do utilizador	Não	Não	NOT NULL
IsAdmin	bit	Flag de administrador	Não	Não	DEFAULT 0
CreatedAt	datetime2	Data de registo	Não	Não	DEFAULT GETDATE()
UpdatedAt	datetime2	Data de última atualização	Não	Não	DEFAULT GETDATE()

Entidade: Books

Atributo	Tipo de Dados	Descrição	Nulos	Multi-valor	Restrição
ISBN	nvarchar(450)	ISBN-13 do livro	Não	Não	PK
Title	nvarchar(max)	Título do livro	Não	Não	NOT NULL
Authors	nvarchar(max)	Autores (separados por vírgula)	Não	Não	NOT NULL
Genre	nvarchar(max)	Género/categoria principal	Não	Não	NOT NULL
Tags	nvarchar(max)	Tags/assuntos (OpenLibrary)	Não	Não	NOT NULL
Description	nvarchar(max)	Sinopse do livro	Não	Não	NOT NULL
Publisher	nvarchar(max)	Editora	Não	Não	NOT NULL
PublicationYear	int	Ano de publicação	Sim	Não	NULL
CoverImageUrl	nvarchar(max)	URL da capa (OpenLibrary)	Não	Não	NOT NULL

Entidade: UserBooks

Atributo	Tipo de Dados	Descrição	Nulos	Multi-valor	Restrição
Id	int (Identity)	Identificador do exemplar	Não	Não	PK
UserId	int	Dono do exemplar	Não	Não	FK → Users
ISBN	nvarchar(450)	Livro a que pertence	Não	Não	FK → Books
Condition	nvarchar(450)	Estado: novo, semi-novo, usado	Não	Não	CHECK, NOT NULL
Quantity	int	Número de exemplares	Não	Não	DEFAULT 1
Available	bit	Disponível para trocas	Não	Não	DEFAULT 1
Emprestado	bit	Indica se está emprestado	Não	Não	DEFAULT 0
IsActive	bit	Soft delete	Não	Não	DEFAULT 1
DateAdded	datetime2	Data de adição	Não	Não	DEFAULT GETDATE()

Entidade: Transactions

Atributo	Tipo de Dados	Descrição	Nulos	Multi-valor	Restrição
Id	int (Identity)	Identificador da transação	Não	Não	PK
Type	nvarchar(max)	Compra / Requisição / Troca	Não	Não	NOT NULL
Status	nvarchar(max)	Estado atual da transação	Não	Não	NOT NULL
Notes	nvarchar(max)	Notas adicionais	Não	Não	NOT NULL
UserBookId	int	Exemplar transacionado (do recetor)	Sim	Não	FK → UserBooks
SenderUserBookId	int	Exemplar oferecido em troca pelo remetente	Sim	Não	FK → UserBooks
SenderId	int	Utilizador remetente	Não	Não	FK → Users
ReceiverId	int	Utilizador destinatário	Não	Não	FK → Users
Price	decimal(18,2)	Preço (para Compra)	Sim	Não	NULL
CreatedAt	datetime2	Data de criação da transação	Não	Não	DEFAULT GETDATE()
UpdatedAt	datetime2	Data da última atualização	Não	Não	DEFAULT GETDATE()

Entidade: Conversations

Atributo	Tipo de Dados	Descrição	Nulos	Multi-valor	Restrição
Id	int (Identity)	Identificador da conversa	Não	Não	PK
InitiatorId	int	Utilizador que iniciou a conversa	Não	Não	FK → Users
ReceiverId	int	Utilizador destinatário inicial	Não	Não	FK → Users
BookISBN	nvarchar(450)	Livro associado (opcional)	Sim	Não	FK → Books, ON DELETE SET NULL
TransactionId	int	Transação associada (opcional)	Sim	Não	FK → Transactions
Status	nvarchar(max)	Active / Archived	Não	Não	CHECK
CreatedAt	datetime2	Data de criação	Não	Não	DEFAULT GETDATE()
LastMessageAt	datetime2	Data da última mensagem	Não	Não	DEFAULT GETDATE()
ArchivedAt	datetime2	Data de arquivamento	Sim	Não	NULL

Entidade: Messages

Atributo	Tipo de Dados	Descrição	Nulos	Multi-valor	Restrição
Id	int (Identity)	Identificador da mensagem	Não	Não	PK
ConversationId	int	Conversa a que pertence	Não	Não	FK → Conversations, ON DELETE CASCADE
SenderId	int	Utilizador autor da mensagem	Não	Não	FK → Users
Content	nvarchar(max)	Conteúdo textual da mensagem	Não	Não	NOT NULL
SentAt	datetime2	Data e hora de envio	Não	Não	DEFAULT GETDATE()
IsRead	bit	Indica se a mensagem foi lida	Não	Não	DEFAULT 0

Entidade: AppSettings

Atributo	Tipo de Dados	Descrição	Nulos	Multi-valor	Restrição
SettingKey	nvarchar(450)	Chave da configuração (nome interno)	Não	Não	PK
SettingValue	nvarchar(max)	Valor da configuração (string)	Não	Não	NOT NULL
Description	nvarchar(max)	Descrição amigável da configuração	Não	Não	NOT NULL
LastModified	datetime2	Data da última alteração do valor	Não	Não	DEFAULT GETDATE()

Entidade: Authors

Atributo	Tipo de Dados	Descrição	Nulos	Multi-valor	Restrição
Id	int (Identity)	Identificador do autor	Não	Não	PK
Name	nvarchar(max)	Nome completo do autor	Não	Não	NOT NULL
Bio	nvarchar(max)	Biografia / descrição (opcional)	Sim	Não	NULL
OpenLibraryId	nvarchar(max)	ID do autor na OpenLibrary	Sim	Não	NULL

Entidade: Categories

Atributo	Tipo de Dados	Descrição	Nulos	Multi-valor	Restrição
Id	int (Identity)	Identificador da categoria	Não	Não	PK
Name	nvarchar(max)	Nome da categoria temática	Não	Não	NOT NULL

Entidade: BookAuthors

Atributo	Tipo de Dados	Descrição	Nulos	Multi-valor	Restrição
BookISBN	nvarchar(450)	ISBN do livro	Não	Não	PK FK → Books, ON DELETE CASCADE
AuthorId	int	Id do autor	Não	Não	PK FK → Authors, ON DELETE CASCADE

Entidade: BookCategories

Atributo	Tipo de Dados	Descrição	Nulos	Multi-valor	Restrição
BookISBN	nvarchar(450)	ISBN do livro	Não	Não	PK FK → Books, ON DELETE CASCADE
CategoryId	int	Id da categoria	Não	Não	PK FK → Categories, ON DELETE CASCADE

Entidade: AuthTokens

Atributo	Tipo de Dados	Descrição	Nulos	Multi-valor	Restrição
Id	int (Identity)	Identificador do token	Não	Não	PK
UserId	int	Utilizador a que o token pertence	Não	Não	FK → Users
TokenHash	nvarchar(max)	Hash do token (não é guardado em claro)	Não	Não	NOT NULL
CreatedAt	datetime2	Data de emissão da sessão	Não	Não	NOT NULL
ExpiresAt	datetime2	Data de expiração da sessão	Não	Não	NOT NULL

2.5. Identificação de Chaves Primárias e Candidatas

Nesta etapa, determinam-se os atributos que permitem identificar de forma única cada registo em cada tabela, distinguindo a Chave Primária das restantes Chaves Candidatas.

Tabela	Chaves Candidatas	Chave Primária	Observações
Users	Id, Email	Id (int)	Email é candidata por ser único, mas utiliza-se Id (inteiro gerado por Identity) para maior eficiência nas relações.
Books	ISBN	ISBN (nvarchar)	O ISBN-13 é internacionalmente único para cada livro. Optou-se por usá-lo como PK para evitar surrogates desnecessários e facilitar a integração com a API OpenLibrary.
UserBooks	Id, (UserId+ISBN+Condition)	Id (int)	A combinação UserId+ISBN+Condition forma um índice único, mas Id é a PK por simplicidade de referência em FK.
Transactions	Id	Id (int)	Identificador único para o registo histórico de cada transação.
Conversations	Id	Id (int)	Identificador da conversa. O par (InitiatorId, ReceiverId, BookISBN) não é único porque podem existir múltiplas conversas sobre o mesmo livro.
Messages	Id	Id (int)	Cada mensagem é identificada por um inteiro gerado por Identity.
AppSettings	SettingKey	SettingKey (nvarchar(450))	A chave é o próprio nome da configuração, tornando o acesso direto e legível. Não existem outras chaves candidatas.
Authors	Id	Id (int)	Identificador único gerado por Identity. O par (Name, OpenLibraryId) poderia atuar como chave alternativa numa fase posterior, mas não está ainda formalizado como UNIQUE.
Categories	Id, Name	Id (int)	Identificador único gerado por Identity. O Name pode constituir chave alternativa (UNIQUE) numa futura iteração.
BookAuthors	(BookISBN, AuthorId)	(BookISBN, AuthorId)	PK composta pelas duas FKs, conforme a Regra 6 do Doc01 — Atividade 3. Resolve a relação muitos-para-muitos entre Books e Authors.
BookCategories	(BookISBN, CategoryId)	(BookISBN, CategoryId)	PK composta pelas duas FKs, conforme a Regra 6 do Doc01 — Atividade 3. Resolve a relação muitos-para-muitos entre Books e Categories.
AuthTokens	Id, TokenHash	Id (int)	Id é a PK por convenção. TokenHash é candidata por ser único na prática (o sistema gera hashes inviáveis de colisionar) e funciona como chave de lookup principal durante a autenticação.

2.6. Diagrama do Modelo Concetual Final

Apresenta-se abaixo o Diagrama Entidade-Relacionamento (DER) final do sistema BookSwap IPKA, construído segundo a notação utilizada nas aulas (pé-de-galinha / crow's foot). Cada entidade é representada por uma caixa contendo o seu nome no

cabeçalho e a lista dos respetivos atributos, com indicação explícita de Chave Primária (PK), Chave Estrangeira (FK) e chaves compostas (PK FK) — utilizadas nas tabelas associativas BookAuthors e BookCategories, conforme a Regra 6 do Doc01 — Atividade 3. As chaves primárias surgem sublinhadas. As cardinalidades de cada relacionamento são representadas pelos símbolos de pé-de-galinha nas pontas das linhas — barras paralelas para o lado “1..1”, pé-de-galinha com círculo para o lado “0..*” e barra com círculo para o lado “0..1” (relações opcionais). O verbo do relacionamento aparece sobre a linha, com um pequeno triângulo a indicar o sentido de leitura.

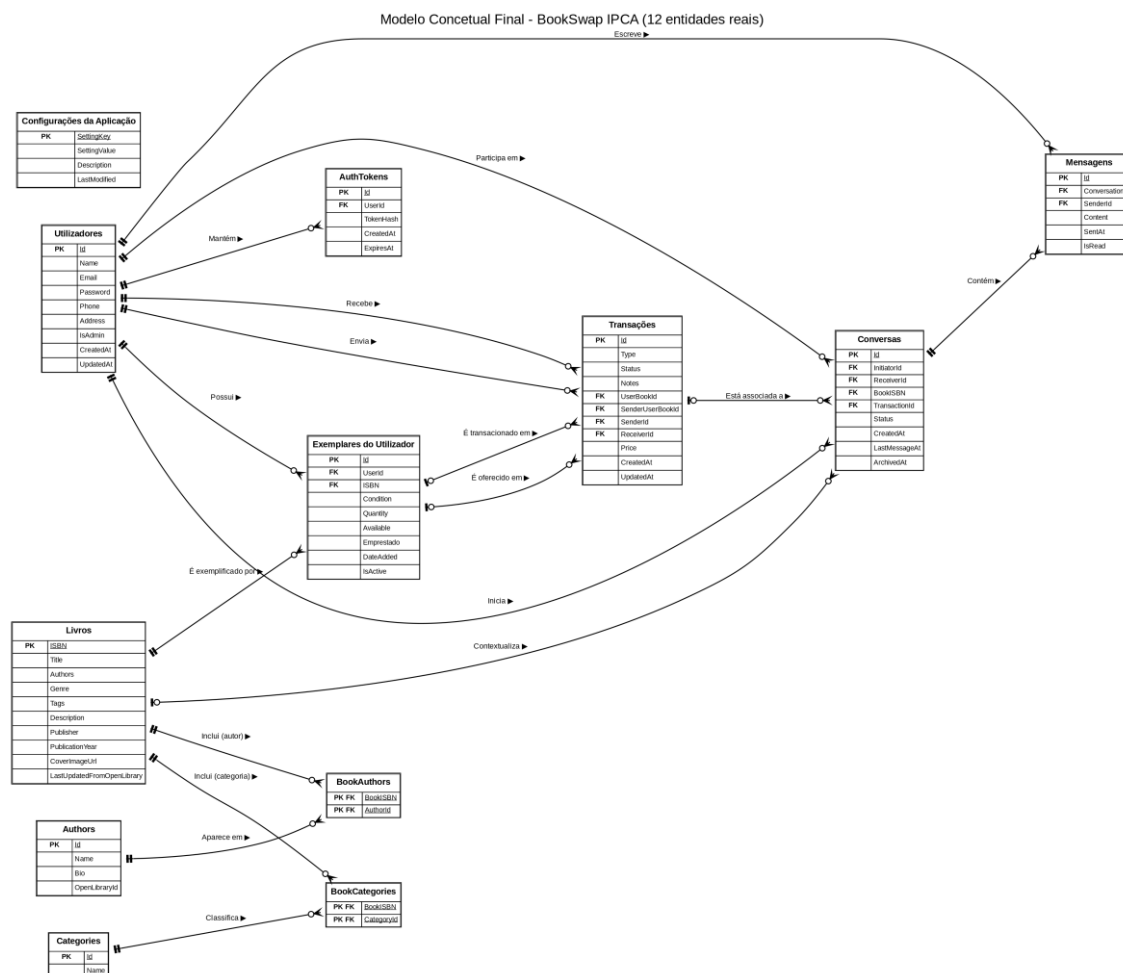


Figura 1 — Diagrama Entidade-Relacionamento (DER) Final — BookSwap IPCA (12 entidades implementadas)

Nota sobre normalização: a desnormalização anterior do atributo Authors (string separada por vírgulas) na entidade Books foi RESOLVIDA na nova arquitetura através da entidade Authors e da tabela associativa BookAuthors, restaurando a 1FN para esta relação. Da mesma forma, o anterior atributo Genre foi normalizado através das entidades Categories e BookCategories. O atributo Tags em Books mantém-se ainda como cadeia de caracteres separada por vírgulas — trata-se de uma desnormalização controlada, justificada pelo facto de as tags virem da API OpenLibrary nesse formato e serem usadas exclusivamente em operações de pesquisa por texto (LIKE/CONTAINS). A sua normalização poderá ser realizada numa fase posterior.

3. Desenho Lógico da Base de Dados

3.1. Derivação dos Relacionamentos para o Modelo Lógico

Nesta fase, cada relacionamento identificado no modelo conceitual é analisado e convertido para o modelo relacional, definindo as chaves estrangeiras e as tabelas de ligação necessárias.

3.1.1. Relações 1:N (Um para Muitos)

Nas relações 1:N, a chave primária da entidade do lado "1" é propagada como chave estrangeira para a tabela do lado "N":

Relacionamento	Regra Aplicada
Users (1) — UserBooks (N)	Adicionar UserId como FK em UserBooks. Um utilizador pode ter múltiplos exemplares; cada exemplar pertence a um único utilizador.
Books (1) — UserBooks (N)	Adicionar ISBN como FK em UserBooks. Um livro pode ter múltiplos exemplares registados por utilizadores diferentes.
Users (1) — AuthTokens (N)	Adicionar UserId como FK em AuthTokens. Um utilizador pode manter múltiplas sessões ativas em simultâneo (ON DELETE CASCADE).
Books (1) — BookAuthors (N)	Adicionar BookISBN como FK em BookAuthors (parte da PK composta). Resolução M:N — Regra 6 do Doc01 — Atividade 3.
Authors (1) — BookAuthors (N)	Adicionar AuthorId como FK em BookAuthors (parte da PK composta). Resolução M:N — Regra 6 do Doc01 — Atividade 3.
Books (1) — BookCategories (N)	Adicionar BookISBN como FK em BookCategories (parte da PK composta). Resolução M:N — Regra 6 do Doc01 — Atividade 3.
Categories (1) — BookCategories (N)	Adicionar CategoryId como FK em BookCategories (parte da PK composta). Resolução M:N — Regra 6 do Doc01 — Atividade 3.
UserBooks (1) — Transactions (N)	Adicionar UserBookId como FK em Transactions, identificando o exemplar transacionado.
Users (1) — Transactions (N) [Sender]	Adicionar SenderId como FK em Transactions, identificando o utilizador remetente.
Users (1) — Transactions (N) [Receiver]	Adicionar ReceiverId como FK em Transactions, identificando o utilizador destinatário.
Users (1) — Conversations (N) [Initiator]	Adicionar InitiatorId como FK em Conversations.
Users (1) — Conversations (N) [Receiver]	Adicionar ReceiverId como FK em Conversations.
Conversations (1) — Messages (N)	Adicionar ConversationId como FK em Messages (ON DELETE CASCADE). Uma conversa contém múltiplas mensagens.

Users (1) —
Messages (N)
[Sender]

Adicionar SenderId como FK em Messages, identificando o autor de cada mensagem.

3.1.2. Relações Opcionais (0..1 para N)

Algumas tabelas possuem referências opcionais (FKs nullable) que permitem associações flexíveis sem forçar a integridade:

- Conversations.BookISBN → Books.ISBN: permite associar uma conversa a um livro específico. A relação é opcional (NULL) pois podem existir conversas não associadas a nenhum livro.
- Conversations.TransactionId → Transactions.Id: permite associar uma conversa a uma transação em curso. Também opcional.
- Transactions.UserBookId → UserBooks.Id: ligação opcional ao exemplar transacionado. É nullable porque transações em estado inicial podem ainda não ter o exemplar concreto definido.
- Transactions.SenderUserBookId → UserBooks.Id: utilizado apenas em trocas diretas para registrar o exemplar oferecido pelo remetente. Nulo nas restantes transações (compras e requisições).

3.1.3. Tratamento de Conflitos de Cascade

Na implementação em SQL Server, o Entity Framework Core deteta potenciais conflitos em caminhos de cascade múltiplos ("multiple cascade paths"). Para as tabelas Transactions e Conversations, as FKs para Users foram configuradas com ON DELETE NO ACTION (para evitar ciclos de cascade), enquanto que as tabelas associativas BookAuthors e BookCategories utilizam ON DELETE CASCADE em ambas as FKs (BookISBN e AuthorId / CategoryId) — quando um livro ou autor/categoria é eliminado, as ligações desaparecem automaticamente. Da mesma forma, AuthTokens.UserId tem ON DELETE CASCADE — eliminar um utilizador remove automaticamente todas as suas sessões.

3.1.4. Índice Único Composto em UserBooks

Para garantir que um utilizador não registre o mesmo livro com o mesmo estado de conservação mais do que uma vez, foi criado um índice único composto sobre (UserId, ISBN, Condition). Esta restrição garante a integridade do catálogo pessoal de cada utilizador.

3.1.5. Relações M:N (Muitos para Muitos)

Conforme estipulado na aula 03 (Modelo Relacional) e na Regra 6 do mapeamento classe→tabela (Doc01 — Atividade 3), as relações muitos-para-muitos devem ser evitadas no modelo final, sendo resolvidas através da criação de uma tabela associativa (também designada por tabela de ligação ou entidade composta). Na nova arquitetura existem duas relações M:N, ambas resolvidas com tabelas associativas com PK composta pelas duas FKs:

- Authors ↔ Books (M:N) — resolvida pela tabela BookAuthors, com PK composta (BookISBN, AuthorId). Cada linha da tabela indica que um determinado autor escreveu um determinado livro. Ambas as FKs têm ON DELETE CASCADE.
- Categories ↔ Books (M:N) — resolvida pela tabela BookCategories, com PK composta (BookISBN, CategoryId). Cada linha indica que um determinado livro

pertence a uma determinada categoria. Ambas as FKs têm ON DELETE CASCADE.

Estas resoluções confirmam a aplicação prática da regra ensinada pela professora: "M:N devem ser evitados — usar tabela de ligação". A nova arquitetura elimina assim qualquer violação da 1FN nos atributos Authors e Genre que existiam anteriormente em Books.

3.2. Validação do Modelo com Normalização

Para validar a robustez da base de dados, aplicou-se o processo de normalização até à Terceira Forma Normal (3FN). Como documento base de análise, utilizou-se um Recibo de Transação Finalizada, que agrega informação de várias entidades.

3.2.1. Documento Não Normalizado (0FN)

Imagine-se um documento 'Recibo de Transação' com todos os atributos num único registo plano:

```
ID_Transacao, Data_Transacao, Tipo_Transacao, Status_Transacao, Preco, Notas, ISBN_Livro, Titulo_Livro, Autores_Livro, Genero_Livro, Descricao_Livro, Editora_Livro, Ano_Publicacao, ID_Remetente, Nome_Remetente, Email_Remetente, ID_Destinatario, Nome_Destinatario, Email_Destinatario, ISBN_Livro_Troca
```

3.2.2. Primeira Forma Normal (1FN)

Condição: não existem grupos repetitivos; todos os atributos são atômicos; existe uma chave primária definida.

- Ação: identificou-se ID_Transacao como chave primária. Quase todos os atributos são atômicos; as exceções eram Autores_Livro e Genero_Livro (que armazenavam listas separadas por vírgulas, violando a 1FN), assim como Tags_Livro. Na arquitetura final, Autores_Livro foi normalizado através das entidades Authors e BookAuthors, e Genero_Livro foi normalizado através de Categories e BookCategories — restaurando a 1FN para ambos os atributos. Apenas Tags permanece como cadeia de caracteres, por opção de desnormalização controlada (origem na API OpenLibrary e uso exclusivo em pesquisa textual).
- Resultado: após a normalização das entidades Authors e Categories, a tabela Books encontra-se na 1FN nos seus atributos essenciais. O campo Tags mantém-se como desnormalização justificada na nota da secção 2.6.

3.2.3. Segunda Forma Normal (2FN)

Condição: está na 1FN e todos os atributos não-chave dependem totalmente da chave primária completa (sem dependências parciais).

Análise das dependências funcionais:

- ID_Transacao → Data_Transacao, Tipo_Transacao, Status_Transacao, Preco, Notas (dependência total — correto).
- ID_Transacao → ID_Remetente, ID_Destinatario (dependência total — correto).
- ISBN_Livro → Titulo_Livro, Descricao_Livro, Editora_Livro, Ano_Publicacao (dependência parcial — estes atributos não dependem de ID_Transacao, mas sim do ISBN). Autores_Livro e Genero_Livro deixaram de ser atributos diretos: passam a ser obtidos pelas tabelas BookAuthors↔Authors e BookCategories↔Categories.

- ID_Remetente → Nome_Remetente, Email_Remetente (dependência transitiva a resolver na 3FN).

Ação: separar as dependências parciais em tabelas independentes.

Tabelas resultantes após 2FN:

- Tabela_Transacoes: (Id, Tipo, Status, Notas, UserBookId [FK→UserBooks], SenderUserBookId [FK→UserBooks], SenderId [FK→Users], ReceiverId [FK→Users], Price, CreatedAt, UpdatedAt)
- Tabela_Books: (ISBN, Title, Description, Publisher, PublicationYear, CoverImageUrl, Tags). Authors e Categories foram extraídas para tabelas próprias, ligadas a Books através das tabelas associativas BookAuthors e BookCategories (relações M:N).

3.2.4. Terceira Forma Normal (3FN)

Condição: está na 2FN e não existem dependências transitivas (um campo não-chave que dependa de outro campo não-chave).

Análise das dependências transitivas restantes:

- ID_Remetente → Nome_Remetente, Email_Remetente: o nome e email do remetente não dependem de ID_Transacao, mas sim de ID_Remetente. Dependência transitiva presente.
- ID_Destinatario → Nome_Destinatario, Email_Destinatario: idem.

Quadro resumo das dependências transitivas detetadas e respetivas tabelas resultantes na 3FN (estilo Connolly & Begg):

Atributo transitivo	Dependência transitiva	Tabela resultante (3FN)
Nome_Remetente, Email_Remetente	Dependem de ID_Remetente (não de ID_Transacao)	Tabela_Users (Id PK, Name, Email, Password, Phone, Address, IsAdmin, CreatedAt, UpdatedAt)
Nome_Destinatario, Email_Destinatario	Dependem de ID_Destinatario (não de ID_Transacao)	Tabela_Users (mesmo modelo acima)
Autores_Livro	Conjunto multivalor que depende de ISBN_Livro — viola 1FN	Tabelas Authors + BookAuthors (M:N), conforme Regra 6 do Doc01 — Atividade 3
Genero_Livro	Categoria que depende de ISBN_Livro — viola 1FN	Tabelas Categories + BookCategories (M:N), conforme Regra 6 do Doc01 — Atividade 3
Title_Livro, Description_Livro, Publisher_Livro, Ano_Publicacao	Atributos do livro que dependem de ISBN_Livro (não de ID_Transacao)	Tabela_Books (ISBN PK, Title, Description, Publisher, PublicationYear, CoverImageUrl, Tags)

Ação: isolar os dados dos utilizadores numa tabela própria.

Tabelas resultantes após 3FN (modelo final):

- Tabela_Users: (Id, Name, Email, Password, Phone, Address, IsAdmin, CreatedAt)

- Tabela_Books: (ISBN, Title, Description, Publisher, PublicationYear, CoverImageUrl, Tags) — sem Authors nem Genre, agora geridos pelas tabelas Authors+BookAuthors e Categories+BookCategories.
- Tabela_Transactions: (Id, Type, Status, Notes, UserBookId [FK→UserBooks], SenderUserBookId [FK→UserBooks, opcional para trocas], SenderId [FK→Users], ReceiverId [FK→Users], Price, CreatedAt, UpdatedAt).

Conclusão: o modelo resultante da normalização até à 3FN coincide com o diagrama lógico apresentado no ponto 3.1, validando a estrutura da base de dados. Acresce que, na nova arquitetura, as tabelas adicionais (UserBooks, Transactions, Conversations, Messages, AppSettings, Authors, Categories, BookAuthors, BookCategories e AuthTokens) já se encontram naturalmente na 3FN desde a sua conceção, pois todos os atributos não-chave dependem estritamente das respetivas chaves primárias, sem dependências parciais ou transitivas. Em particular, as tabelas associativas BookAuthors e BookCategories — que aplicam a Regra 6 do Doc01 — não introduzem qualquer redundância, contendo apenas as duas FKs que formam a sua PK composta.

3.2.5. Tabela de Normalização — Resumo Visual

Forma Normal	Alterações Efetuadas	Justificação
1FN	Definição de PK (ID_Transacao). Verificação da atomicidade de todos os atributos.	Eliminação de grupos repetitivos. Garantia de valores atómicos por campo.
2FN	Criação das tabelas independentes Livros e Utilizadores, separando os dados bibliográficos e pessoais.	O título, autores e género do livro dependem do ISBN, não do ID da transação — dependência parcial eliminada.
3FN	Separação total dos atributos descritivos dos utilizadores (nome, email) para a tabela Users.	Eliminação de dependências transitivas: Nome_Remetente dependia de ID_Remetente, não de ID_Transacao.

3.3. Validação do Modelo com Transações do Utilizador

Nesta fase, validou-se a eficácia do modelo relacional ao testar se a estrutura de dados permite responder a questões reais de utilização da plataforma BookSwap. Para cada transação apresenta-se o percurso dos dados e a expressão em Álgebra Relacional.

(a) Listar todos os livros disponíveis e o seu dono

Objetivo: verificar se a relação de posse entre utilizadores e o catálogo de exemplares está corretamente mapeada, incluindo a obtenção dos autores (via BookAuthors+Authors) e categorias (via BookCategories+Categories) de cada livro.

Percurso: Users → UserBooks ← Books → BookAuthors → Authors; Books → BookCategories → Categories

Álgebra Relacional:

```

π Users.Name, Books.Title, Authors.Name AS Autor, Categories.Name AS
Categoria, UserBooks.Condition
(σ UserBooks.Available=1 ∧ UserBooks.IsActive=1
  (UserBooks ⋈ UserBooks.UserId=Users.Id Users
    ⋈ UserBooks.ISBN=Books.ISBN Books
      ⋈ BookAuthors.BookISBN=Books.ISBN BookAuthors
      ⋈ BookAuthors.AuthorId=Authors.Id Authors
  )
)

```

```
⋈ BookCategories.BookISBN=Books.ISBN BookCategories
⋈ BookCategories.CategoryId=Categories.Id Categories))
```

(b) Listar todos os livros de um determinado autor

Objetivo: validar a relação M:N entre Books e Authors, verificando que é possível obter todos os livros escritos por um autor específico através da tabela associativa BookAuthors.

Percurso: Authors → BookAuthors → Books

Álgebra Relacional:

```
π Books.ISBN, Books.Title, Books.Publisher, Books.PublicationYear
(σ Authors.Id=@AuthorId
 (Authors ⋈ Authors.Id=BookAuthors.AuthorId BookAuthors
 ⋈ BookAuthors.BookISBN=Books.ISBN Books))
```

(c) Listar o histórico de transações de um utilizador

Objetivo: verificar se o histórico de trocas, compras e requisições está acessível através das relações.

Percurso: Users → Transactions ← UserBooks ← Books (via UserBookId)

Álgebra Relacional:

```
π Transactions.Id, Transactions.Type, Transactions.Status, Books.Title,
Users_R.Name AS Destinatario
(σ Transactions.SenderId=@UserId
 (Transactions ⋈ Transactions.UserBookId=UserBooks.Id UserBooks
 ⋈ UserBooks.ISBN=Books.ISBN Books
```

```
⋈ Transactions.ReceiverId=Users_R.Id ρ Users_R (Users)))
```

(d) Obter todas as conversas ativas de um utilizador com últimas mensagens

Objetivo: validar o sistema de mensagens — se é possível obter as conversas e as mensagens associadas a partir de um utilizador.

Percurso: Users → Conversations ← Messages (última mensagem)

Álgebra Relacional (simplificada):

```
π Conversations.Id, Users_R.Name, Books.Title, Conversations.LastMessageAt
(σ Conversations.Status='Active' ∧
 (Conversations.InitiatorId=@UserId ∨ Conversations.ReceiverId=@UserId)
 (Conversations ⋈ Books ⋈ Users_R))
```

(e) Listar mensagens enviadas por um utilizador num intervalo de tempo

Objetivo: validar a relação Users→Messages (Escreve) — verificar que é possível obter todas as mensagens individuais enviadas por um utilizador, ordenadas por data.

Percurso: Users → Messages → Conversations (para o contexto da conversa)

Álgebra Relacional:

```
π Messages.Id, Messages.Content, Messages.SentAt, Conversations.Id,
Users_R.Name AS Destinatario
(σ Messages.SenderId=@UserId ∧ Messages.SentAt BETWEEN @inicio AND
@fim
 (Messages ⋈ Messages.ConversationId=Conversations.Id Conversations
 ⋈ (Conversations.ReceiverId OR Conversations.InitiatorId)=Users_R.Id ρ
Users_R (Users)))
```

(f) Listar sessões ativas de um determinado utilizador

Objetivo: validar a relação Users→AuthTokens — verificar que é possível obter todas as sessões ativas (tokens não expirados) de um utilizador, ordenadas por data de criação.

Percurso: Users → AuthTokens (filtrado por ExpiresAt > NOW)

Álgebra Relacional:

```

$$\pi \text{ AuthTokens.Id, AuthTokens.CreatedAt, AuthTokens.ExpiresAt } \\ (\sigma \text{ AuthTokens.UserId=@UserId} \wedge \text{AuthTokens.ExpiresAt} > \text{@agora} \\ (\text{AuthTokens}))$$

```

3.4. Diagrama do Modelo Lógico Final

Segundo a aula 02 (Modelos de Dados) da unidade curricular, o esquema relacional é a representação visual das entidades, atributos e relacionamentos de uma base de dados. O modelo lógico final do BookSwap resulta da aplicação de todas as transformações descritas nas secções anteriores: propagação das chaves primárias como chaves estrangeiras nas tabelas do lado “N” das relações 1:N; resolução das relações muitos-para-muitos Livros↔Authors e Livros↔Categories através das tabelas associativas BookAuthors e BookCategories (Regra 6 do Doc01 — Atividade 3), eliminando a desnormalização anterior dos atributos Authors e Genre da entidade Books; referências opcionais (nullable) nas FKs BookISBN e TransactionId de Conversations e nos campos UserBookId e SenderUserBookId de Transactions; e integração da nova tabela AuthTokens para gestão de sessões.

Nota: o diagrama abaixo coincide com o apresentado em 2.6 (Figura 1) porque, no nosso caso, o modelo concetual final já incluía a lista completa de atributos e a marcação explícita de chaves primárias e estrangeiras. Esta coincidência é coerente com o que a professora ensina na aula 02 — o esquema relacional combina entidades, atributos e relacionamentos numa só representação. A Figura 2 é a confirmação visual de que a estrutura lógica obtida pela normalização (3.2) e validada pelas operações em álgebra relacional (3.3) corresponde exatamente ao modelo concetual proposto.

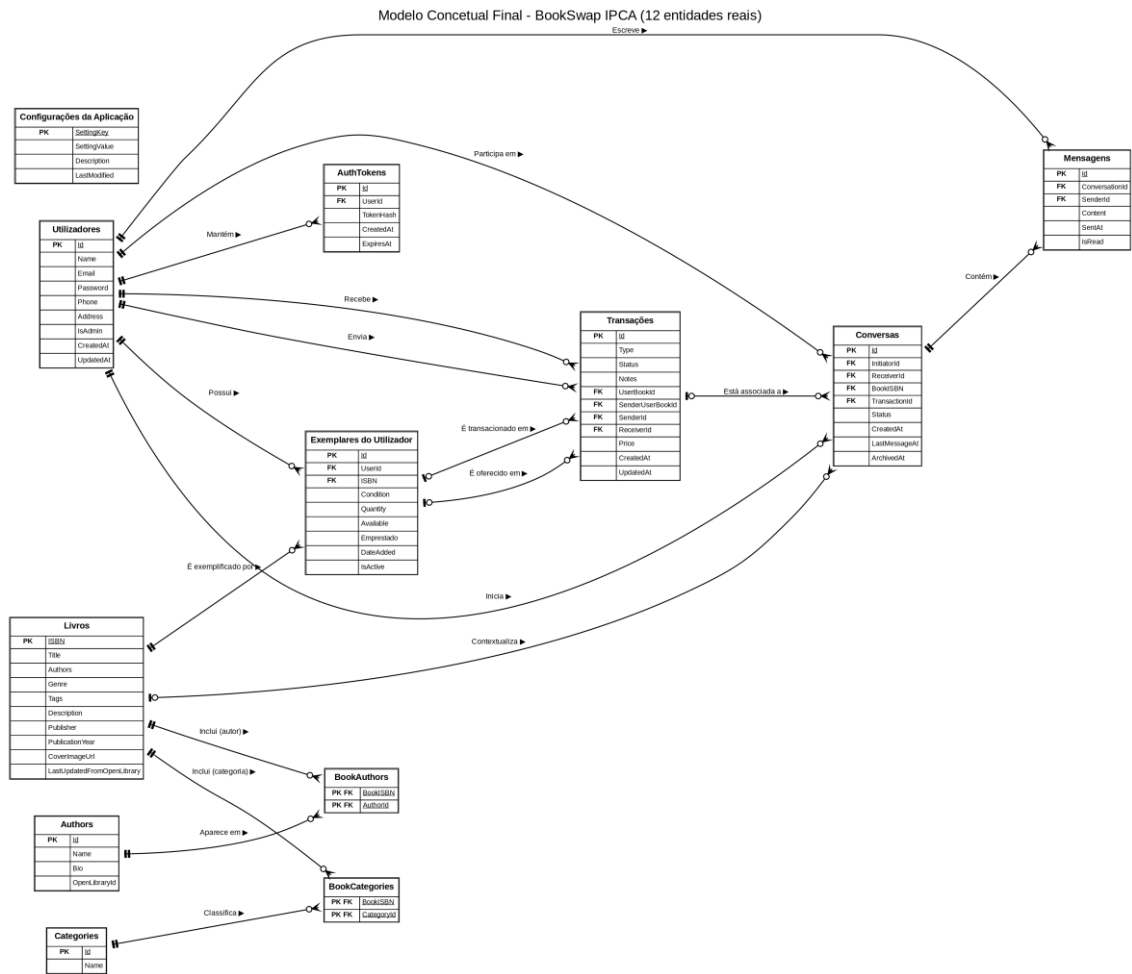


Figura 2 — Modelo Lógico Final em notação pé-de-galinha (crow's foot) — BookSwap IPCA (12 entidades)

4. Desenho Físico da Base de Dados

4.1. Tradução do Modelo Lógico para o Modelo Físico

Nesta etapa, procedeu-se à conversão das entidades lógicas para a sua implementação física no SGBD SQL Server. Para cada atributo, foram definidos os tipos de dados adequados, as chaves primárias e estrangeiras, as restrições de integridade e a obrigatoriedade de preenchimento.

Tabela	Atributo	Tipo de Dados	Chave	R
Users	Id	int	PK	IDENTITY (1,1)
	Name	nvarchar(max)		—
	Email	nvarchar(max)		UNIQUE
	Password	nvarchar(max)		BCrypt hash
	Phone	nvarchar(max)		—
	Address	nvarchar(max)		—
	IsAdmin	bit		DEFAULT(0)
Books	CreatedAt	datetime2		DEFAULT(GETUTCDATE())
	UpdatedAt	datetime2		DEFAULT(GETUTCDATE())
	ISBN	nvarchar(450)	PK	ISBN-13
	Title	nvarchar(max)		—
	Authors	nvarchar(max)		legado (string CSV)
	Genre	nvarchar(max)		legado (string única)
	Tags	nvarchar(max)		Separadas por vírgula
	Description	nvarchar(max)		—
	Publisher	nvarchar(max)		—
	PublicationYear	int		—
UserBooks	CoverImageUrl	nvarchar(max)		URL OpenLibrary
	LastUpdatedFromOpenLibrary	datetime2		DEFAULT(GETUTCDATE())
	Id	int	PK	IDENTITY(1,1)
	UserId	int	FK	REFERENCES Users(Id) ON DELETE
	ISBN	nvarchar(450)	FK	REFERENCES Books(ISBN) ON DELETE
	Condition	nvarchar(450)		CHECK(condition IN('novo','sem
	Quantity	int		DEFAULT(1), CHECK(quantity > 0
	Available	bit		DEFAULT(1)
	Emprestado	bit		DEFAULT(0)
	DateAdded	datetime2		DEFAULT(GETUTCDATE())
	IsActive	bit		DEFAULT(1) — soft delete
Transactions	Id	int	PK	IDENTITY(1,1)

Conversations	Type	nvarchar(max)		CHECK(IN('Compra','Requisição'))
	Status	nvarchar(max)		CHECK(IN('Pendente','Aceita',''))
	Notes	nvarchar(max)		DEFAULT('')
	UserBookId	int	FK	REFERENCES UserBooks(Id) NO ACTION
	SenderUserBookId	int	FK	REFERENCES UserBooks(Id) NO ACTION
	SenderId	int	FK	REFERENCES Users(Id) NO ACTION
	ReceiverId	int	FK	REFERENCES Users(Id) NO ACTION
	Price	decimal(18,2)		CHECK(price > 0)
	CreatedAt	datetime2		DEFAULT(GETUTCDATE())
	UpdatedAt	datetime2		DEFAULT(GETUTCDATE())
Messages	Id	int	PK	IDENTITY(1,1)
	InitiatorId	int	FK	REFERENCES Users(Id) NO ACTION
	ReceiverId	int	FK	REFERENCES Users(Id) NO ACTION
	BookISBN	nvarchar(450)	FK	REFERENCES Books(ISBN) SET NULL
	TransactionId	int	FK	REFERENCES Transactions(Id) NO ACTION
	Status	nvarchar(max)		CHECK(IN('Active','Archived'))
	CreatedAt	datetime2		DEFAULT(GETUTCDATE())
	LastMessageAt	datetime2		DEFAULT(GETUTCDATE())
	ArchivedAt	datetime2		—
	Id	int	PK	IDENTITY(1,1)
AppSettings	ConversationId	int	FK	REFERENCES Conversations(Id) NO ACTION
	SenderId	int	FK	REFERENCES Users(Id) NO ACTION
	Content	nvarchar(max)		—
	SentAt	datetime2		DEFAULT(GETUTCDATE())
	IsRead	bit		DEFAULT(0)
	SettingKey	nvarchar(450)	PK	—
	SettingValue	nvarchar(max)		—
	Description	nvarchar(max)		—
	LastModified	datetime2		DEFAULT(GETUTCDATE())
	Id	int	PK	IDENTITY(1,1)
Authors	Name	nvarchar(max)		—
	Bio	nvarchar(max)		—
	OpenLibraryId	nvarchar(max)		ID na OpenLibrary
	Id	int	PK	IDENTITY(1,1)
Categories	Name	nvarchar(max)		—
	BookISBN	nvarchar(450)	PK FK	REFERENCES Books(ISBN) CASCADE
BookAuthors	AuthorId	int	PK FK	REFERENCES Authors(Id) CASCADE
	BookISBN	nvarchar(450)	PK FK	REFERENCES Books(ISBN) CASCADE

AuthTokens	CategoryId	int	PK FK	REFERENCES Categories(Id) CASC
	Id	int	PK	IDENTITY(1,1)
	UserId	int	FK	REFERENCES Users(Id) CASCADE
	TokenHash	nvarchar(max)		—
	CreatedAt	datetime2		DEFAULT(GETUTCDATE())
	ExpiresAt	datetime2		—

4.2. Índices

Para otimizar a performance das consultas, foram definidos os seguintes índices:

Índices por Chave Primária (Clustered) — criados automaticamente pelo SQL Server:

- PK_Users: coluna Id da tabela Users.
- PK_Books: coluna ISBN da tabela Books.
- PK_UserBooks: coluna Id da tabela UserBooks.
- PK_Transactions: coluna Id da tabela Transactions.
- PK_Conversations: coluna Id da tabela Conversations.
- PK_Messages: coluna Id da tabela Messages.
- PK_AppSettings: coluna SettingKey da tabela AppSettings.
- PK_Authors: coluna Id da tabela Authors.
- PK_Categories: coluna Id da tabela Categories.
- PK_BookAuthors: PK composta (BookISBN, AuthorId) na tabela BookAuthors — Regra 6 do Doc01 — Atividade 3.
- PK_BookCategories: PK composta (BookISBN, CategoryId) na tabela BookCategories — Regra 6 do Doc01 — Atividade 3.
- PK_AuthTokens: coluna Id da tabela AuthTokens.

Índices por Chave Estrangeira (Non-Clustered) — a criar explicitamente:

- IX_UserBooks_UserId: em UserBooks(UserId) — otimiza a listagem dos livros de um utilizador.
- IX_UserBooks_ISBN: em UserBooks(ISBN) — otimiza a pesquisa de exemplares por ISBN.
- IX_UserBooks_Unique: UNIQUE em UserBooks(UserId, ISBN, Condition) — impede duplicados.
- IX_Transactions_UserBookId: em Transactions(UserBookId) — otimiza a consulta de transações por exemplar (chave estrangeira para UserBooks).
- IX_Transactions_SenderId: em Transactions(SenderId) — otimiza o histórico de envios.
- IX_Transactions_ReceiverId: em Transactions(ReceiverId) — otimiza o histórico de receções.
- IX_Conversations_InitiatorId: em Conversations(InitiatorId).
- IX_Conversations_ReceiverId: em Conversations(ReceiverId).
- IX_Messages_ConversationId: em Messages(ConversationId) — essencial para carregar mensagens.
- IX_Messages_SenderId: em Messages(SenderId).

- IX_Transactions_SenderUserBookId: em Transactions(SenderUserBookId) — otimiza a consulta de exemplares oferecidos em trocas.
- IX_BookAuthors_AuthorId: em BookAuthors(AuthorId) — otimiza a obtenção de todos os livros de um autor.
- IX_BookCategories_CategoryId: em BookCategories(CategoryId) — otimiza a obtenção de todos os livros de uma categoria.
- IX_AuthTokens_UserId: em AuthTokens(UserId) — otimiza a obtenção das sessões ativas de um utilizador (essencial em cada autenticação).

Índices de unicidade e pesquisa de texto:

- UQ_Users_Email: índice único em Users(Email) — impede registos duplicados e acelera o login.
- IX_Books_Title: índice em Books(Title) — suporta pesquisa rápida por título na barra de busca.

4.3. Scripts SQL de Criação da Base de Dados

Apresentam-se abaixo os scripts T-SQL para a criação completa da base de dados BookSwap no SQL Server.

```
-- =====
-- SCRIPT DE CRIAÇÃO DA BASE DE DADOS BOOKSWAP IPCA
-- SGBD: Microsoft SQL Server
-- Versão: 1.0 | Data: Maio 2026
-- =====

USE master;
GO

IF EXISTS (SELECT name FROM sys.databases WHERE name = 'BookSwapDB')
    DROP DATABASE BookSwapDB;
GO

CREATE DATABASE BookSwapDB
    COLLATE Latin1_General_CI_AS;
GO

USE BookSwapDB;
GO

-- =====
-- TABELA: Users
-- =====
CREATE TABLE Users (
    Id            INT IDENTITY(1,1) NOT NULL,
    Name          NVARCHAR(MAX)     NOT NULL,
    Email         NVARCHAR(MAX)     NOT NULL,
    Password      NVARCHAR(MAX)     NOT NULL, -- BCrypt hash
    Phone        NVARCHAR(MAX)     NOT NULL,
    Address       NVARCHAR(MAX)     NOT NULL,
    IsAdmin       BIT               NOT NULL DEFAULT(0),
    CreatedAt     DATETIME2         NOT NULL DEFAULT(GETUTCDATE()),
    UpdatedAt     DATETIME2         NOT NULL DEFAULT(GETUTCDATE()),
    CONSTRAINT PK_Users PRIMARY KEY (Id)
);
GO

CREATE UNIQUE INDEX UQ_Users_Email ON Users(Email);
GO
```

```

-- =====
-- TABELA: Books
-- =====
CREATE TABLE Books (
    ISBN                NVARCHAR(450) NOT NULL,
    Title               NVARCHAR(MAX) NOT NULL,
    Authors             NVARCHAR(MAX) NOT NULL,
    Genre               NVARCHAR(MAX) NOT NULL,
    Tags                NVARCHAR(MAX) NOT NULL DEFAULT(''),
    Description          NVARCHAR(MAX) NOT NULL DEFAULT(''),
    Publisher            NVARCHAR(MAX) NOT NULL DEFAULT(''),
    PublicationYear      INT           NULL,
    CoverImageUrl       NVARCHAR(MAX) NOT NULL DEFAULT(''),
    LastUpdatedFromOpenLibrary DATETIME2 NOT NULL DEFAULT(GETUTCDATE()),
    CONSTRAINT PK_Books PRIMARY KEY (ISBN)
);
GO

CREATE INDEX IX_Books_Title ON Books(Title);
GO

-- =====
-- TABELA: UserBooks
-- =====
CREATE TABLE UserBooks (
    Id                  INT IDENTITY(1,1) NOT NULL,
    UserId              INT              NOT NULL,
    ISBN                NVARCHAR(450)    NOT NULL,
    Condition            NVARCHAR(450)    NOT NULL,
    Quantity            INT              NOT NULL DEFAULT(1),
    Available           BIT              NOT NULL DEFAULT(1),
    Emprestado          BIT              NOT NULL DEFAULT(0),
    IsActive            BIT              NOT NULL DEFAULT(1),
    DateAdded           DATETIME2        NOT NULL DEFAULT(GETUTCDATE()),
    CONSTRAINT PK_UserBooks PRIMARY KEY (Id),
    CONSTRAINT FK_UB_User FOREIGN KEY (UserId) REFERENCES Users(Id) ON
DELETE CASCADE,
    CONSTRAINT FK_UB_Book FOREIGN KEY (ISBN) REFERENCES Books(ISBN)
ON DELETE CASCADE,
    CONSTRAINT CK_UB_Condition CHECK (Condition IN ('novo', 'semi-novo',
'usado')),
    CONSTRAINT CK_UB_Quantity CHECK (Quantity > 0)
);
GO

CREATE UNIQUE INDEX UQ_UserBooks_UserISBNCondition ON UserBooks(UserId,
ISBN, Condition);
CREATE INDEX IX_UserBooks_UserId ON UserBooks(UserId);
CREATE INDEX IX_UserBooks_ISBN ON UserBooks(ISBN);
GO

-- =====
-- TABELA: Transactions
-- =====
CREATE TABLE Transactions (
    Id                  INT IDENTITY(1,1) NOT NULL,
    Type               NVARCHAR(MAX) NOT NULL,
    Status             NVARCHAR(MAX) NOT NULL DEFAULT('Pendente'),
    Notes              NVARCHAR(MAX) NOT NULL DEFAULT(""),
    UserBookId         INT           NULL, -- Exemplar transacionado
    SenderUserBookId   INT           NULL, -- Exemplar oferecido em trocas
    SenderId           INT           NOT NULL,

```

```

ReceiverId      INT          NOT NULL,
Price           DECIMAL(18,2) NULL,
CreatedAt       DATETIME2    NOT NULL DEFAULT(GETUTCDATE()),
UpdatedAt       DATETIME2    NOT NULL DEFAULT(GETUTCDATE()),
CONSTRAINT PK_Transactions PRIMARY KEY (Id),
CONSTRAINT FK_T_UserBook FOREIGN KEY (UserBookId)
REFERENCES UserBooks(Id) ON DELETE NO ACTION,
CONSTRAINT FK_T_SenderUB FOREIGN KEY (SenderUserBookId)
REFERENCES UserBooks(Id) ON DELETE NO ACTION,
CONSTRAINT FK_T_Sender FOREIGN KEY (SenderId) REFERENCES
Users(Id) ON DELETE NO ACTION,
CONSTRAINT FK_T_Receiver FOREIGN KEY (ReceiverId) REFERENCES
Users(Id) ON DELETE NO ACTION,
CONSTRAINT CK_T_Type CHECK (Type IN ('Compra','Requisição','Troca')),
CONSTRAINT CK_T_Status CHECK (Status IN
('Pendente','Aceita','Confirmada','Rejeitada','Cancelada')),
CONSTRAINT CK_T_Price CHECK (Price IS NULL OR Price > 0),
CONSTRAINT CK_T_DiffUsers CHECK (SenderId <> ReceiverId)
);
GO
CREATE INDEX IX_Trans_UserBookId ON Transactions(UserBookId);
CREATE INDEX IX_Trans_SenderUserBookId ON
Transactions(SenderUserBookId);
CREATE INDEX IX_Trans_Sender ON Transactions(SenderId);
CREATE INDEX IX_Trans_Receiver ON Transactions(ReceiverId);
GO
-- TABELA: Conversations
-- =====
CREATE TABLE Conversations (
    Id INT IDENTITY(1,1) NOT NULL,
    InitiatorId INT NOT NULL,
    ReceiverId INT NOT NULL,
    BookISBN NVARCHAR(450) NULL,
    TransactionId INT NULL,
    Status NVARCHAR(MAX) NOT NULL DEFAULT('Active'),
    CreatedAt DATETIME2 NOT NULL DEFAULT(GETUTCDATE()),
    LastMessageAt DATETIME2 NOT NULL DEFAULT(GETUTCDATE()),
    ArchivedAt DATETIME2 NULL,
    CONSTRAINT PK_Conversations PRIMARY KEY (Id),
    CONSTRAINT FK_C_Initiator FOREIGN KEY (InitiatorId) REFERENCES
Users(Id),
    CONSTRAINT FK_C_Receiver FOREIGN KEY (ReceiverId) REFERENCES
Users(Id),
    CONSTRAINT FK_C_Book FOREIGN KEY (BookISBN) REFERENCES
Books(ISBN) ON DELETE SET NULL,
    CONSTRAINT FK_C_Transaction FOREIGN KEY (TransactionId) REFERENCES
Transactions(Id),
    CONSTRAINT CK_C_Status CHECK (Status IN ('Active','Archived')),
    CONSTRAINT CK_C_DiffUsers CHECK (InitiatorId <> ReceiverId)
);
GO

CREATE INDEX IX_Conv_InitiatorId ON Conversations(InitiatorId);
CREATE INDEX IX_Conv_ReceiverId ON Conversations(ReceiverId);
GO

-- =====
-- TABELA: Messages
-- =====
CREATE TABLE Messages (
    Id INT IDENTITY(1,1) NOT NULL,

```

```

        ConversationId INT NOT NULL,
        SenderId INT NOT NULL,
        Content NVARCHAR(MAX) NOT NULL,
        SentAt DATETIME2 NOT NULL DEFAULT(GETUTCDATE()),
        IsRead BIT NOT NULL DEFAULT(0),
        CONSTRAINT PK_Messages PRIMARY KEY (Id),
        CONSTRAINT FK_M_Convo FOREIGN KEY (ConversationId) REFERENCES
Conversations(Id) ON DELETE CASCADE,
        CONSTRAINT FK_M_Sender FOREIGN KEY (SenderId) REFERENCES
Users(Id)
);
GO

CREATE INDEX IX_Msg_ConversationId ON Messages(ConversationId);
CREATE INDEX IX_Msg_SenderId ON Messages(SenderId);
GO

```

```

-- =====
-- TABELA: AppSettings
-- =====
CREATE TABLE AppSettings (
    SettingKey NVARCHAR(450) NOT NULL,
    SettingValue NVARCHAR(MAX) NOT NULL,
    Description NVARCHAR(MAX) NOT NULL,
    LastModified DATETIME2 NOT NULL DEFAULT(GETUTCDATE()),
    CONSTRAINT PK_AppSettings PRIMARY KEY (SettingKey)
);
GO

-- =====
-- TABELA: Authors
-- =====
CREATE TABLE Authors (
    Id INT IDENTITY(1,1) NOT NULL,
    Name NVARCHAR(MAX) NOT NULL,
    Bio NVARCHAR(MAX) NULL,
    OpenLibraryId NVARCHAR(MAX) NULL,
    CONSTRAINT PK_Authors PRIMARY KEY (Id)
);
GO

-- =====
-- TABELA: Categories
-- =====
CREATE TABLE Categories (
    Id INT IDENTITY(1,1) NOT NULL,
    Name NVARCHAR(MAX) NOT NULL,
    CONSTRAINT PK_Categories PRIMARY KEY (Id)
);
GO

-- =====
-- TABELA: BookAuthors (junção M:N Books ↔ Authors)
-- Regra 6 do Doc01 — Atividade 3: PK composta pelas duas FKs
-- =====
CREATE TABLE BookAuthors (
    BookISBN NVARCHAR(450) NOT NULL,

```

```

    AuthorId INT          NOT NULL,
    CONSTRAINT PK_BookAuthors PRIMARY KEY (BookISBN, AuthorId),
    CONSTRAINT FK_BA_Book  FOREIGN KEY (BookISBN) REFERENCES
Books(ISBN) ON DELETE CASCADE,
    CONSTRAINT FK_BA_Author FOREIGN KEY (AuthorId) REFERENCES
Authors(Id) ON DELETE CASCADE
);
GO
CREATE INDEX IX_BookAuthors_AuthorId ON BookAuthors(AuthorId);
GO

-- =====
-- TABELA: BookCategories (junção M:N Books ↔ Categories)
-- Regra 6 do Doc01 — Atividade 3: PK composta pelas duas FKs
-- =====
CREATE TABLE BookCategories (
    BookISBN NVARCHAR(450) NOT NULL,
    CategoryId INT          NOT NULL,
    CONSTRAINT PK_BookCategories PRIMARY KEY (BookISBN, CategoryId),
    CONSTRAINT FK_BC_Book  FOREIGN KEY (BookISBN) REFERENCES
Books(ISBN) ON DELETE CASCADE,
    CONSTRAINT FK_BC_Category FOREIGN KEY (CategoryId) REFERENCES
Categories(Id) ON DELETE CASCADE
);
GO
CREATE INDEX IX_BookCategories_CategoryId ON BookCategories(CategoryId);
GO

-- =====
-- TABELA: AuthTokens
-- =====
CREATE TABLE AuthTokens (
    Id INT IDENTITY(1,1) NOT NULL,
    UserId INT          NOT NULL,
    TokenHash NVARCHAR(MAX) NOT NULL,
    CreatedAt DATETIME2 NOT NULL DEFAULT(GETUTCDATE()),
    ExpiresAt DATETIME2 NOT NULL,
    CONSTRAINT PK_AuthTokens PRIMARY KEY (Id),
    CONSTRAINT FK_AT_User  FOREIGN KEY (UserId) REFERENCES Users(Id)
ON DELETE CASCADE
);
GO
CREATE INDEX IX_AuthTokens_UserId ON AuthTokens(UserId);
GO

```

4.4. Scripts SQL de Inserção de Dados

Apresentam-se abaixo os scripts T-SQL de inserção de dados de teste, que permitem verificar o correto funcionamento de todas as restrições e relações da base de dados.

```

-- =====
-- SCRIPT DE INSERÇÃO DE DADOS DE TESTE — BOOKSWAP IPCA
-- =====

USE BookSwapDB;

```

GO

```
-- Inserção de Utilizadores (passwords são hashes BCrypt de '123456')
INSERT INTO Users (Name, Email, Password, Phone, Address, IsAdmin) VALUES
('Marta Silva', 'marta@alunos.ipca.pt', '$2a$11$hXfM...hashBCrypt',
'912345678', 'Barcelos', 0),
('João Santos', 'joao@alunos.ipca.pt', '$2a$11$hXfM...hashBCrypt',
'913456789', 'Braga', 0),
('Ana Ferreira', 'ana@alunos.ipca.pt', '$2a$11$hXfM...hashBCrypt',
'914567890', 'Guimarães', 0),
('Carlos Oliveira', 'carlos@alunos.ipca.pt', '$2a$11$hXfM...hashBCrypt',
NULL, NULL, 0),
('Admin IPCA', 'admin@ipca.pt', '$2a$11$hXfM...hashBCrypt',
'253 802 000', 'IPCA', 1);
GO
```

```
-- Inserção de Livros (dados típicos da API OpenLibrary)
INSERT INTO Books (ISBN, Title, Authors, Genre, Tags, Description,
Publisher, PublicationYear, CoverImageUrl) VALUES
('9780132350884', 'Clean Code', 'Robert C. Martin', 'Informática',
'programming, software engineering, best practices',
'Um guia de boas práticas de programação orientado a objetos.',
'Prentice Hall', 2008,
'https://covers.openlibrary.org/b/isbn/9780132350884-L.jpg'),
('9780201633610', 'Design Patterns', 'Erich Gamma, Richard Helm',
'Informática',
'design patterns, object-oriented, software',
'Padrões de design reutilizáveis para software orientado a objetos.',
'Addison-Wesley', 1994,
'https://covers.openlibrary.org/b/isbn/9780201633610-L.jpg'),
('9789724066264', 'Fundamentos de Bases de Dados', 'Fernanda Lima', 'Bases
de Dados',
'sql, databases, relational, normalization',
'Manual universitário sobre conceção e implementação de bases de
dados.',
'FCA', 2014, ''),
('9780062316097', 'The Pragmatic Programmer', 'David Thomas, Andrew Hunt',
'Informática',
'programming, pragmatic, software craft',
'Conselhos práticos para programadores modernos.',
'Addison-Wesley', 2019,
'https://covers.openlibrary.org/b/isbn/9780062316097-L.jpg');
GO
```

```
-- Inserção de Exemplares (UserBooks)
INSERT INTO UserBooks (UserId, ISBN, Condition, Quantity, Available) VALUES
(1, '9780132350884', 'semi-novo', 1, 1), -- Marta tem Clean Code semi-novo
(1, '9780201633610', 'usado', 1, 1), -- Marta tem Design Patterns
usado
(2, '9789724066264', 'novo', 2, 1), -- João tem 2 exemplares novos de
BD
(3, '9780132350884', 'novo', 1, 0), -- Ana tem Clean Code novo,
indisponível
(4, '9780062316097', 'semi-novo', 1, 1); -- Carlos tem Pragmatic Programmer
GO
```

-- Inserção de Transações

```
INSERT INTO Transactions (Type, Status, Notes, UserBookId, SenderUserBookId,
SenderId, ReceiverId, Price) VALUES
('Requisição', 'Confirmada', 'Empréstimo de Design Patterns por 2 semanas.',
2, NULL, 1, 3, NULL), -- Marta empresta exemplar de Design Patterns à Ana
('Troca', 'Pendente', 'Troca de Clean Code por outro exemplar.',
1, 3, 1, 2, NULL), -- Marta oferece Clean Code à João, recebendo outro
UserBook
('Compra', 'Confirmada', 'Compra direta de Fundamentos de BD.',
```



```

3, NULL, 2, 4, 15.00); -- João vende o seu exemplar (UserBook 3) ao Carlos por
15€
GO
-- Inserção de Conversas e Mensagens
INSERT INTO Conversations (InitiatorId, ReceiverId, BookISBN, Status) VALUES
(2, 1, '9780132350884', 'Active'); -- João iniciou conversa com Marta
sobre Clean Code

INSERT INTO Messages (ConversationId, SenderId, Content) VALUES
(1, 2, 'Olá Marta! O Clean Code ainda está disponível?'),
(1, 1, 'Olá João! Sim, ainda está. Queres vir buscar?'),
(1, 2, 'Ótimo! Podemos combinar para amanhã na biblioteca.');
```

GO

```

-- AppSettings: configurações iniciais
INSERT INTO AppSettings (SettingKey, SettingValue, Description) VALUES
('AllowDuplicateBooks', 'false', 'Permite que um utilizador registe múltiplos
exemplares do mesmo livro na mesma condição. '),
('ShowSingleBookCondition', 'true', 'Mostra a condição do livro quando o utilizador
tem apenas um exemplar. ');
GO
```

```

-- Authors: catálogo de autores
INSERT INTO Authors (Name, Bio, OpenLibraryId) VALUES
('Robert C. Martin', 'Engenheiro de software conhecido como Uncle Bob.',
'OL27695A'),
('Eça de Queirós', 'Escritor português do séc. XIX.', 'OL5739184A'),
('Andrew S. Tanenbaum', 'Professor e autor de obras clássicas em redes.',
'OL30168A'),
('C. J. Date', 'Autor de obras de referência em bases de dados.', 'OL1450A');
GO
```

```

-- Categories: catálogo de categorias temáticas
INSERT INTO Categories (Name) VALUES
('Informática'),
('Engenharia de Software'),
('Romance'),
('Redes de Computadores'),
('Bases de Dados');
GO
```

```

-- BookAuthors: ligações livro–autor (M:N)
INSERT INTO BookAuthors (BookISBN, AuthorId) VALUES
('9780132350884', 1), -- Clean Code — Robert C. Martin
('9780201633610', 1), -- Design Patterns — Robert C. Martin (co-autoria)
('9789724066264', 4); -- Fundamentos de BD — C. J. Date
GO
```

```

-- BookCategories: ligações livro–categoria (M:N)
INSERT INTO BookCategories (BookISBN, CategoryId) VALUES
('9780132350884', 1), -- Clean Code — Informática
('9780132350884', 2), -- Clean Code — Engenharia de Software (duas categorias)
('9780201633610', 2), -- Design Patterns — Engenharia de Software
('9789724066264', 5); -- Fundamentos de BD — Bases de Dados
GO
```

```
-- AuthTokens: sessões iniciais de exemplo
INSERT INTO AuthTokens (UserId, TokenHash, ExpiresAt) VALUES
(1, '$2a$11$tokenHashExemplo1.....', DATEADD(DAY, 7, GETUTCDATE())),
(3, '$2a$11$tokenHashExemplo2.....', DATEADD(DAY, 7, GETUTCDATE()));
GO
```

4.5. Stored Procedures

As Stored Procedures implementadas visam encapsular a lógica de negócio no servidor de base de dados, garantindo a integridade transacional e facilitando a manutenção.

SP_RegisterUserBook — Registo de Exemplar

Esta procedure verifica se o utilizador e o livro existem antes de inserir um novo exemplar. Impede duplicados com base no estado de conservação.

```
CREATE PROCEDURE SP_RegisterUserBook
    @UserId INT,
    @ISBN NVARCHAR(450),
    @Condition NVARCHAR(450),
    @Quantity INT = 1
AS
BEGIN
    SET NOCOUNT ON;
    IF NOT EXISTS (SELECT 1 FROM Users WHERE Id = @UserId)
    BEGIN
        RAISERROR('Erro: Utilizador não encontrado.', 16, 1); RETURN;
    END
    IF NOT EXISTS (SELECT 1 FROM Books WHERE ISBN = @ISBN)
    BEGIN
        RAISERROR('Erro: Livro (ISBN) não encontrado no catálogo.', 16, 1);
    RETURN;
    END
    IF EXISTS (SELECT 1 FROM UserBooks WHERE UserId=@UserId AND ISBN=@ISBN
    AND Condition=@Condition AND IsActive=1)
    BEGIN
        RAISERROR('Já tens um exemplar deste livro com esse estado de
    conservação.', 16, 1); RETURN;
    END
    INSERT INTO UserBooks (UserId, ISBN, Condition, Quantity, Available,
    IsActive)
    VALUES (@UserId, @ISBN, @Condition, @Quantity, 1, 1);
    PRINT 'Exemplar registado com sucesso. ID: ' + CAST(SCOPE_IDENTITY() AS
    NVARCHAR);
    END;
    GO
```

SP_ProcessTransaction — Processar Transação com Controlo de Estado

Procedure crítica que atualiza o estado de uma transação e, quando confirmada, marca o exemplar como indisponível. Utiliza controlo transacional com TRY/CATCH.

```
CREATE PROCEDURE SP_ProcessTransaction
    @TransactionId INT,
    @NewStatus NVARCHAR(50),
    @ExecutorId INT -- Id do utilizador que executa a ação
AS
BEGIN
    SET NOCOUNT ON;
    DECLARE @CurrentStatus NVARCHAR(50);
    DECLARE @UserBookId INT;
    DECLARE @SenderId INT;
    SELECT @CurrentStatus=Status, @UserBookId=UserBookId, @SenderId=SenderId
    FROM Transactions WHERE Id=@TransactionId;
```

```

IF @CurrentStatus IS NULL
BEGIN
    RAISERROR('Transação não encontrada.', 16, 1); RETURN;
END
-- Validar transições de estado permitidas
IF NOT (@CurrentStatus='Pendente' AND @NewStatus IN
('Aceita','Rejeitada','Cancelada'))
    AND NOT (@CurrentStatus='Aceita' AND @NewStatus IN
('Confirmada','Cancelada'))
BEGIN
    RAISERROR('Transição de estado inválida.', 16, 1); RETURN;
END
BEGIN TRANSACTION;
BEGIN TRY
    UPDATE Transactions SET Status=@NewStatus, UpdatedAt=GETUTCDATE()
    WHERE Id=@TransactionId;
    -- Se confirmada, marcar exemplar como indisponível
    IF @NewStatus = 'Confirmada'
    BEGIN
        UPDATE UserBooks SET Available=0
        UPDATE UserBooks SET Available=0
    END

    COMMIT TRANSACTION;
    PRINT 'Transação atualizada para: ' + @NewStatus;
END TRY
BEGIN CATCH
    ROLLBACK TRANSACTION;
    RAISERROR('Erro ao processar transação: %s', 16, 1,
    ERROR_MESSAGE());
END CATCH
END;
GO

```

SP_GetUserStats — Estatísticas de Utilizador

Procedure que retorna um resumo estatístico do perfil de um utilizador: total de livros, trocas realizadas, trocas pendentes e conversas ativas.

```

CREATE PROCEDURE SP_GetUserStats
    @UserId INT
AS
BEGIN
    SET NOCOUNT ON;
    SELECT
        U.Name,
        U.Email,
        (SELECT COUNT(*) FROM UserBooks WHERE UserId=@UserId AND IsActive=1)
    AS TotalLivros,
        (SELECT COUNT(*) FROM UserBooks WHERE UserId=@UserId AND Available=1
    AND IsActive=1) AS LivrosDisponiveis,
        (SELECT COUNT(*) FROM Transactions WHERE SenderId=@UserId AND
    Status='Confirmada') AS TrocasRealizadas,
        (SELECT COUNT(*) FROM Transactions WHERE (SenderId=@UserId OR
    ReceiverId=@UserId) AND Status='Pendente') AS TransacoesPendentes,
        (SELECT COUNT(*) FROM Conversations WHERE (InitiatorId=@UserId OR
    ReceiverId=@UserId) AND Status='Active') AS ConversasAtivas
    FROM Users U WHERE U.Id = @UserId;
END;
GO

```

4.6. Funções (UDF)

As funções definidas pelo utilizador (UDF) foram implementadas para simplificar cálculos repetitivos e reutilizá-los em consultas e vistas.

FN_CountUserBooks — Contagem de Exemplos Ativos

```
CREATE FUNCTION FN_CountUserBooks (@UserId INT)
RETURNS INT
AS
BEGIN
    DECLARE @Total INT;
    SELECT @Total = COUNT(*) FROM UserBooks
    WHERE UserId = @UserId AND IsActive = 1;
    RETURN ISNULL(@Total, 0);
END;
GO
```

FN_GetBookAvailabilityLabel — Estado de Disponibilidade Formatado

```
CREATE FUNCTION FN_GetBookAvailabilityLabel (@UserBookId INT)
RETURNS NVARCHAR(30)
AS
BEGIN
    DECLARE @Available BIT;
    DECLARE @Emprestado BIT;
    SELECT @Available=Available, @Emprestado=Emprestado
    FROM UserBooks WHERE Id=@UserBookId;
    RETURN CASE
        WHEN @Emprestado = 1 THEN 'Emprestado'
        WHEN @Available = 1 THEN 'Disponível'
        ELSE 'Indisponível'
    END;
END;
GO
```

FN_GetConditionLabel — Tradução do Estado de Conservação

```
CREATE FUNCTION FN_GetConditionLabel (@Condition NVARCHAR(50))
RETURNS NVARCHAR(30)
AS
BEGIN
    RETURN CASE @Condition
        WHEN 'novo' THEN 'Novo'
        WHEN 'semi-novo' THEN 'Semi-Novo'
        WHEN 'usado' THEN 'Usado'
        ELSE 'Desconhecido'
    END;
END;
GO
```

FN_HasPendingTransaction — Verificar Transação Pendente Duplicada

```
CREATE FUNCTION FN_HasPendingTransaction (@UserId INT, @UserBookId INT)
RETURNS BIT
AS
BEGIN
    DECLARE @Exists BIT = 0;
    IF EXISTS (
        SELECT 1 FROM Transactions
        WHERE (SenderId=@UserId OR ReceiverId=@UserId)
        AND UserBookId=@UserBookId AND Status='Pendente'
    ) SET @Exists = 1;
    RETURN @Exists;
END;
GO
```

4.7. Vistas (Views)

As vistas foram implementadas para simplificar o acesso a dados relacionados, ocultando a complexidade das junções e fornecendo uma interface mais amigável para a camada de aplicação.

VW_AvailableBooks — Catálogo de Exemplares Disponíveis

Apresenta todos os exemplares disponíveis para troca, com informação do livro e do dono.

```
CREATE VIEW VW_AvailableBooks AS
SELECT
    UB.Id                AS ExemplarId,
    B.ISBN,
    B.Title              AS Titulo,
    B.Authors            AS Autores,
    B.Genre              AS Genero,
    B.CoverImageUrl     AS CapaURL,
    UB.Condition         AS EstadoConservacao,
    UB.Quantity          AS Quantidade,
    U.Id                AS DonoId,
    U.Name              AS DonoNome,
    UB.DateAdded        AS DataAdicionado
FROM UserBooks UB
INNER JOIN Books B    ON UB.ISBN = B.ISBN
INNER JOIN Users U    ON UB.UserId = U.Id
WHERE UB.Available = 1 AND UB.IsActive = 1 AND UB.Emprestado = 0;
GO
```

VW_PendingTransactions — Gestão de Transações Pendentes

Vista para administradores e donos de exemplares verificarem as transações pendentes (pedidos, trocas e compras em estado 'Pendente').

```
CREATE VIEW VW_PendingTransactions AS
SELECT
    T.Id                AS TransacaoId,
    T.Type              AS Tipo,
    B.Title             AS TituloLivro,
    B.ISBN,
    U_Send.Name         AS Remetente,
    U_Recv.Name         AS Destinatario,
    UB.Condition        AS Estado,
    T.CreatedAt         AS DataPedido,
    T.Status            AS EstadoTransacao
FROM Transactions T
INNER JOIN UserBooks UB    ON T.UserBookId = UB.Id
INNER JOIN Books B        ON UB.ISBN = B.ISBN
INNER JOIN Users U_Send   ON T.SenderId = U_Send.Id
INNER JOIN Users U_Recv   ON T.ReceiverId = U_Recv.Id
WHERE T.Status = 'Pendente';
GO
```

VW_UserStatistics — Estatísticas Agregadas por Utilizador

```
CREATE VIEW VW_UserStatistics AS
SELECT
    U.Id,
    U.Name              AS Nome,
    U.Email,
    U.CreatedAt        AS MemberDesde,
    dbo.FN_CountUserBooks(U.Id) AS
TotalExemplares,
    (SELECT COUNT(*) FROM Transactions WHERE SenderId=U.Id AND
Status='Confirmada') AS TrocasEnviadas,
```

```

        (SELECT COUNT(*) FROM Transactions WHERE ReceiverId=U.Id AND
Status='Confirmada') AS TrocasRecebidas
FROM Users U
WHERE U.IsAdmin = 0;
GO

```

VW_TransactionHistory — Histórico Completo de Transações

```

CREATE VIEW VW_TransactionHistory AS
SELECT
    T.Id                AS TransacaoId,
    T.Type              AS Tipo,
    T.Status            AS Estado,
    B.Title             AS Livro,
    B.ISBN,
    U_S.Name            AS Remetente,
    U_R.Name            AS Destinatario,
    T.Price             AS Preço,
    T.Notes             AS Notas,
    T.CreatedAt         AS DataCriacao,
    T.UpdatedAt         AS UltimaAtualizacao
FROM Transactions T
INNER JOIN UserBooks UB ON T.UserBookId = UB.Id
INNER JOIN Books B      ON UB.ISBN      = B.ISBN
INNER JOIN Users U_S    ON T.SenderId   = U_S.Id
INNER JOIN Users U_R    ON T.ReceiverId = U_R.Id;
GO

```

4.8. Triggers

Os triggers foram implementados para garantir a automação de processos lógicos e a integridade referencial complexa que não pode ser resolvida apenas com restrições de chave estrangeira.

TRG_UpdateConversationTimestamp — Atualizar Timestamp da Conversa

Após a inserção de uma nova mensagem, este trigger atualiza automaticamente o campo LastMessageAt da conversa correspondente, garantindo a ordenação correta das conversas.

```

CREATE TRIGGER TRG_UpdateConversationTimestamp
ON Messages
AFTER INSERT
AS
BEGIN
    SET NOCOUNT ON;
    UPDATE Conversations
    SET LastMessageAt = GETUTCDATE()
    WHERE Id IN (SELECT DISTINCT ConversationId FROM inserted);
END;
GO

```

TRG_PreventDuplicateTransaction — Prevenir Transações Pendentes Duplicadas

Impede que um utilizador crie mais do que uma transação pendente para o mesmo exemplar (UserBook).

```

CREATE TRIGGER TRG_PreventDuplicateTransaction
ON Transactions
FOR INSERT
AS
BEGIN
    SET NOCOUNT ON;
    IF EXISTS (

```

```

        SELECT 1 FROM Transactions T
        JOIN inserted I ON T.UserBookId = I.UserBookId AND T.SenderId =
I.SenderId
        WHERE T.Status = 'Pendente' AND T.Id <> I.Id
    )
    BEGIN
        ROLLBACK TRANSACTION;
        RAISERROR('Já existe uma transação pendente para este exemplar.',
16, 1);
    END
END;
GO

```

TRG_PreventSelfTransaction — Impedir Auto-Transação

Garante que um utilizador não pode iniciar uma transação consigo mesmo.

```

CREATE TRIGGER TRG_PreventSelfTransaction
ON Transactions
FOR INSERT, UPDATE
AS
BEGIN
    SET NOCOUNT ON;
    IF EXISTS (SELECT 1 FROM inserted WHERE SenderId = ReceiverId)
    BEGIN
        ROLLBACK TRANSACTION;
        RAISERROR('Não é permitido criar uma transação entre o mesmo
utilizador.', 16, 1);
    END
END;
GO

```

TRG_ProtectBookFromDeletion — Proteção de Livros com Histórico

Impede a eliminação de um livro do catálogo se existirem transações associadas (histórico protegido).

```

CREATE TRIGGER TRG_ProtectBookFromDeletion
ON Books
INSTEAD OF DELETE
AS
BEGIN
    SET NOCOUNT ON;
    IF EXISTS (
        SELECT 1 FROM Transactions T
        JOIN UserBooks UB ON T.UserBookId = UB.Id
        JOIN deleted D ON UB.ISBN = D.ISBN
    )
    BEGIN
        RAISERROR('Não é possível eliminar um livro com histórico de
transações.', 16, 1);
        RETURN;
    END
    DELETE FROM Books WHERE ISBN IN (SELECT ISBN FROM deleted);
    PRINT 'Livro removido do catálogo com sucesso.';
END;
GO

```

TRG_MarkMessagesRead — Marcação Automática de Mensagens Lidas

Quando o estado de uma conversa é atualizado para 'Archived', marca automaticamente todas as suas mensagens como lidas.

```

CREATE TRIGGER TRG_ArchiveConversationMessages
ON Conversations

```

```
AFTER UPDATE
AS
BEGIN
    SET NOCOUNT ON;
    IF EXISTS (
        SELECT 1 FROM inserted I JOIN deleted D ON I.Id=D.Id
        WHERE I.Status='Archived' AND D.Status<>'Archived'
    )
    BEGIN
        UPDATE Messages
        SET IsRead = 1
        WHERE ConversationId IN (SELECT Id FROM inserted WHERE
Status='Archived');
    END
END;
GO
```


5. Conclusão

O desenvolvimento da base de dados para a plataforma BookSwap IPCA permitiu consolidar e aplicar de forma integrada os conhecimentos lecionados nas aulas teóricas de Armazenamento e Acesso a Dados, nomeadamente: modelação de dados (aula 02), modelo relacional e suas regras de integridade (aula 03), normalização até à 3ª Forma Normal (aula 04), álgebra relacional (aula 05) e SQL avançado — DDL, DML, stored procedures, funções, vistas e triggers (aulas 06 a 08).

Ao longo das várias etapas do projeto, foi possível transformar uma necessidade real da comunidade estudantil do IPCA — a partilha informal de manuais escolares — numa solução tecnológica estruturada e robusta. O processo começou pela análise de requisitos e modelação concetual (Diagrama Entidade-Relacionamento), evoluiu para o modelo lógico através das transformações descritas pela Regra 6 do Doc01 — Atividade 3 (criação de entidades associativas para resolver relações M:N), e culminou na implementação física em SQL Server.

O modelo final concetual identifica doze entidades, todas implementadas como tabelas na base de dados SQL Server: Users, Books, UserBooks, Transactions, Conversations, Messages, AppSettings, Authors, Categories, BookAuthors, BookCategories e AuthTokens. As tabelas associativas BookAuthors e BookCategories (com PK composta — Regra 6 do Doc01 — Atividade 3) resolvem as relações muitos-para-muitos entre Books↔Authors e Books↔Categories, eliminando a desnormalização inicial dos atributos Authors e Genre que existiam em Books. A nova tabela AuthTokens centraliza a gestão de sessões e autenticação dos utilizadores.

Os objetos de programação implementados (aula 08) desempenham papéis específicos na manutenção do ecossistema:

- As Stored Procedures (SP_RegisterUserBook, SP_ProcessTransaction, SP_GetUserStats) encapsulam a lógica de negócio crítica, protegendo as operações transacionais com controlo TRY/CATCH e validações de estado.
- As Funções (UDF) (FN_CountUserBooks, FN_GetBookAvailabilityLabel, FN_GetConditionLabel, FN_HasPendingTransaction) simplificam cálculos recorrentes e permitem a sua reutilização em vistas e consultas ad hoc.
- As Vistas (VW_AvailableBooks, VW_PendingTransactions, VW_UserStatistics, VW_TransactionHistory) oferecem uma camada de abstração que facilita o desenvolvimento de consultas na camada de aplicação, ocultando a complexidade dos JOINS.
- Os Triggers (TRG_UpdateConversationTimestamp, TRG_PreventDuplicateTransaction, TRG_PreventSelfTransaction, TRG_ProtectBookFromDeletion, TRG_MarkMessagesRead) garantem a automação de processos e a integridade referencial em tempo real, complementando as restrições declarativas (PK, FK, CHECK, UNIQUE).

Em síntese, o projeto demonstra que uma base de dados bem estruturada e normalizada é o alicerce de qualquer aplicação de sucesso. O modelo final do BookSwap responde aos requisitos atuais de gestão de livros, trocas e comunicação entre utilizadores. Como trabalho futuro, o grupo prevê: (i) concluir a implementação da tabela Authors e da respetiva tabela associativa BookAuthors, eliminando a desnormalização do atributo Authors em Books; (ii) definir e implementar a décima entidade do modelo; (iii) avaliar a normalização do atributo Tags em Books, se a evolução do sistema o justificar; e (iv) acrescentar índices de pesquisa textual avançada (full-text search) sobre Title e Authors para melhorar a experiência de procura.

Bibliografia

Materiais da Unidade Curricular

- Martinho, M. (2024). Apresentação da unidade curricular (00-Apresentacao). IPCA — Curso TeSP em Aplicações Móveis, Armazenamento e Acesso a Dados.
- Martinho, M. (2024). Introdução às Bases de Dados (01-IntroducaoBD). IPCA — Curso TeSP em Aplicações Móveis, Armazenamento e Acesso a Dados.
- Martinho, M. (2024). Modelos de Dados (02-ModelosDeDados). IPCA — Curso TeSP em Aplicações Móveis, Armazenamento e Acesso a Dados.
- Martinho, M. (2024). Modelo Relacional (03-ModeloRelacional). IPCA — Curso TeSP em Aplicações Móveis, Armazenamento e Acesso a Dados.
- Martinho, M. (2024). Normalização (04-Normalizacao). IPCA — Curso TeSP em Aplicações Móveis, Armazenamento e Acesso a Dados.
- Martinho, M. (2024). Álgebra Relacional (05-AlgebraRelacional). IPCA — Curso TeSP em Aplicações Móveis, Armazenamento e Acesso a Dados.
- Martinho, M. (2024). SQL — Linguagem de Manipulação de Dados (06-SQL-LMD). IPCA — Curso TeSP em Aplicações Móveis, Armazenamento e Acesso a Dados.
- Martinho, M. (2024). SQL — Linguagem de Definição de Dados (07-SQL-LDD). IPCA — Curso TeSP em Aplicações Móveis, Armazenamento e Acesso a Dados.
- Martinho, M. (2024). SQL Avançado — Stored Procedures, Funções, Vistas e Triggers (08-SQL-Avancado). IPCA — Curso TeSP em Aplicações Móveis, Armazenamento e Acesso a Dados.
- Portela, M. (s.d.). Atividade 3 — Modelo Relacional vs. Diagrama de Classes: Regras de Mapeamento (Doc01). IPCA — Curso TeSP em Aplicações Móveis, Armazenamento e Acesso a Dados.

Livros e Manuais Académicos

- Connolly, T. & Begg, C. (2015). Database Systems: A Practical Approach to Design, Implementation, and Management (6.^a ed.). Pearson.
- Damásio, M. (2017). SQL — Structured Query Language. Lisboa: FCA — Editora de Informática.
- Gouveia, F. (2014). Fundamentos de Bases de Dados. Lisboa: FCA — Editora de Informática.
- Ramakrishnan, R., & Gehrke, J. (2003). Database Management Systems (3.^a ed.). McGraw-Hill.
- Silberschatz, A., Korth, H., & Sudarshan, S. (2020). Database System Concepts (7.^a ed.). McGraw-Hill.

Documentação Técnica Online

- Microsoft Learn. (2024). SQL Server Documentation. Disponível em: <https://learn.microsoft.com/en-us/sql/sql-server/>
- Microsoft Learn. (2024). Entity Framework Core Documentation. Disponível em: <https://learn.microsoft.com/en-us/ef/core/>
- OpenLibrary. (2024). Open Library API. Disponível em: <https://openlibrary.org/developers/api>
- W3Schools. (2024). SQL Tutorial. Disponível em: <https://www.w3schools.com/sql/>

Software e Ferramentas Utilizadas

- Microsoft SQL Server Management Studio (SSMS): implementação física, testes de queries e gestão de objetos de base de dados.
- Entity Framework Core 8.0: ORM utilizado na camada de acesso a dados da API ASP.NET Core 9.
- Draw.io / Lucidchart: ferramentas para a elaboração dos diagramas concetual (ER) e lógico.
- Visual Studio Code: escrita e formatação do código SQL e documentação.
- Flutter / Dart: desenvolvimento da aplicação móvel cliente.